Logistic Regression

Presented by TriNode Team

Binary Classes

From Regression to Classification

Why a straight line fails for yes / no answers

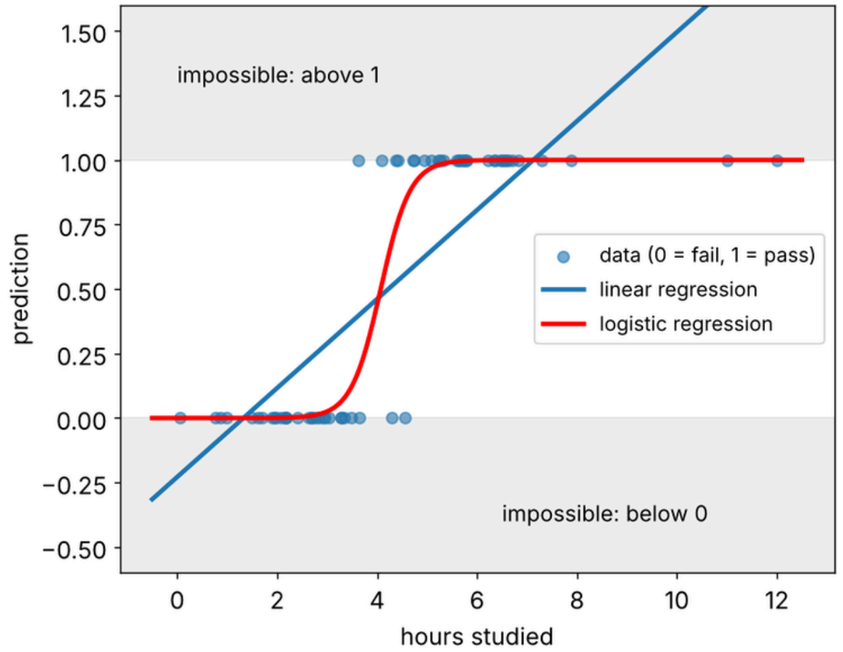
Classification
The target is a category, not a number: pass or fail, spam or not spam, sick or healthy.

Linear regression breaks
A straight line predicts values above 1 and below 0, which cannot be probabilities.

Sensitive to extremes
A few students who studied 12 hours tilt the whole line and move the decision point.

What we need
A model whose output always stays between 0 and 1 and can be read as a probability.

VISUAL



Linear regression (blue) leaves the 0–1 range; logistic regression (red) bends into an S-shape that never does.

Types of Classification Problems

Binary, multiclass and multilabel

Type	Meaning	Examples
Binary	Exactly two classes (0 / 1)	Spam or not, pass or fail, tumor malignant or benign
Multiclass	Three or more classes, one per sample	Iris species, digit 0–9, news topic
Multilabel	Several labels can be true at once	A photo tagged “beach” and “sunset”

Logistic regression handles binary directly, and multiclass with one-vs-rest or softmax

Logistic Regression: The Idea

A linear score turned into a probability

Step 1: a linear score

Compute z = w · x + b exactly like linear regression. z can be any number.

Step 2: squash it

Pass z through the sigmoid σ(z) to get a probability p between 0 and 1.

Step 3: decide

Predict class 1 if p ≥ 0.5, otherwise class 0. The 0.5 threshold can be changed.

Output

The model gives both a probability (how sure) and a class (the decision).

FORMULA

$$p = \sigma(w \cdot x + b) = 1 / (1 + e^{-(w \cdot x + b)})$$

Symbol	Meaning
x	features of one sample
w, b	weights and bias, learned in training
p	probability that y = 1

Example: z = 2 → p = σ(2) = 0.88 → predict class 1

Logistic Regression

The Sigmoid Function

Squashing any number into a probability

Formula

$\sigma(z) = 1 / (1 + e^{-z})$. The output is always strictly between 0 and 1.

Key values

$\sigma(0) = 0.5$, large positive z gives almost 1, very negative z gives almost 0.

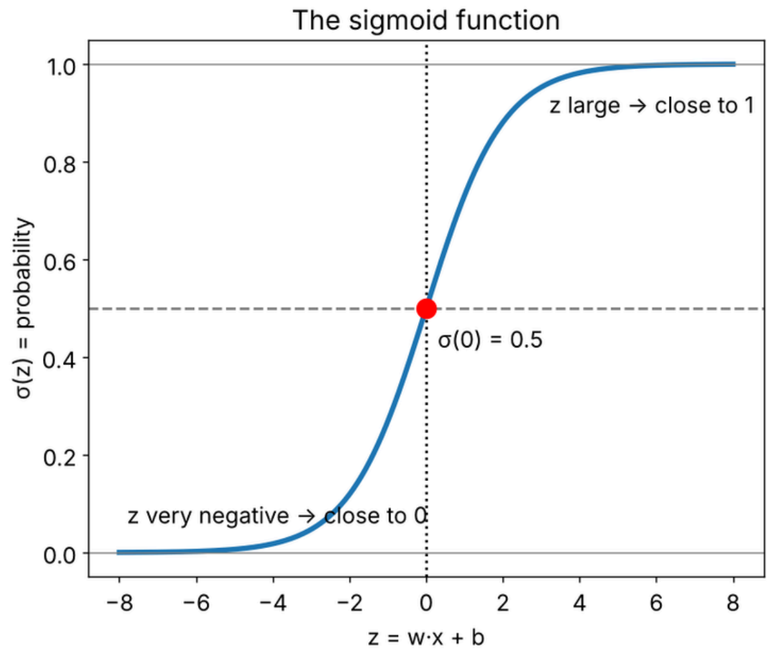
Symmetry

$\sigma(-z) = 1 - \sigma(z)$: the curve is symmetric around the point (0, 0.5).

Easy derivative

$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$, which makes the gradients simple to compute.

VISUAL



The S-shaped sigmoid curve: steep near 0, flat at both ends.

The Sigmoid in Code

One line of NumPy

```
sigmoid(z) = 1 / (1 + np.exp(-z))
```

- Works on single numbers and whole arrays
- Negative scores give probabilities below 0.5
- The symmetry σ(z) + σ(−z) = 1 always holds

ml_demo.py

```python
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

for z in [-5, -2, 0, 2, 5]:
    print(f"z = {z:3}  ->  sigma(z) = {sigmoid(z):.3f}")
print()
print("sigma(2) + sigma(-2) =", sigmoid(2) + sigmoid(-2))
d = sigmoid(0) * (1 - sigmoid(0))
print("slope at z = 0:", d)
```

</> OUTPUT

```
z =  -5  ->  sigma(z) = 0.007
z =  -2  ->  sigma(z) = 0.119
z =   0  ->  sigma(z) = 0.500
z =   2  ->  sigma(z) = 0.881
z =   5  ->  sigma(z) = 0.993
sigma(2) + sigma(-2) = 0.9999999999999999
slope at z = 0: 0.25
```

Odds & Log-Odds

How to read the weights of a logistic model

Odds

odds = p / (1 - p). p = 0.8 gives odds of 4: success is four times as likely as failure.

Log-odds (logit)

Logistic regression is linear in the log-odds: log(p / (1 - p)) = w · x + b.

Reading a weight

Raising x_j by 1 adds w_j to the log-odds, which multiplies the odds by e^{w_j}.

Sign of a weight

w > 0 makes class 1 more likely, w < 0 makes it less likely, w = 0 has no effect.

FORMULA

$$\log\left(\frac{p}{1-p}\right) = w \cdot x + b$$

Symbol	Meaning
p / (1 - p)	the odds of class 1
e^{w_j}	odds ratio for +1 in feature j

Example: w = 0.7 for hours studied → each extra hour multiplies the odds of passing by $e^{0.7} \approx 2$

The Decision Boundary

Where the model switches its answer

Where p = 0.5

p = 0.5 exactly when $z = w \cdot x + b = 0$. That set of points is the decision boundary.

A straight line

With two features the boundary is a line, with three a plane: logistic regression is a linear classifier.

Distance means confidence

Points far from the boundary get probabilities near 0 or 1; points close to it are uncertain.

Curved boundaries

Add polynomial features (x^2, x_1x_2) to let the boundary bend, as in polynomial regression.

FORMULA

$$W_1X_1 + W_2X_2 + b = 0$$

Symbol	Meaning
x_1, x_2	two features, e.g. two exam scores
solve for x_2	$x_2 = -(w_1x_1 + b) / w_2$

Example: Points on one side → class 1, other side → class 0

Decision Boundary in Code

Admission from two exam scores

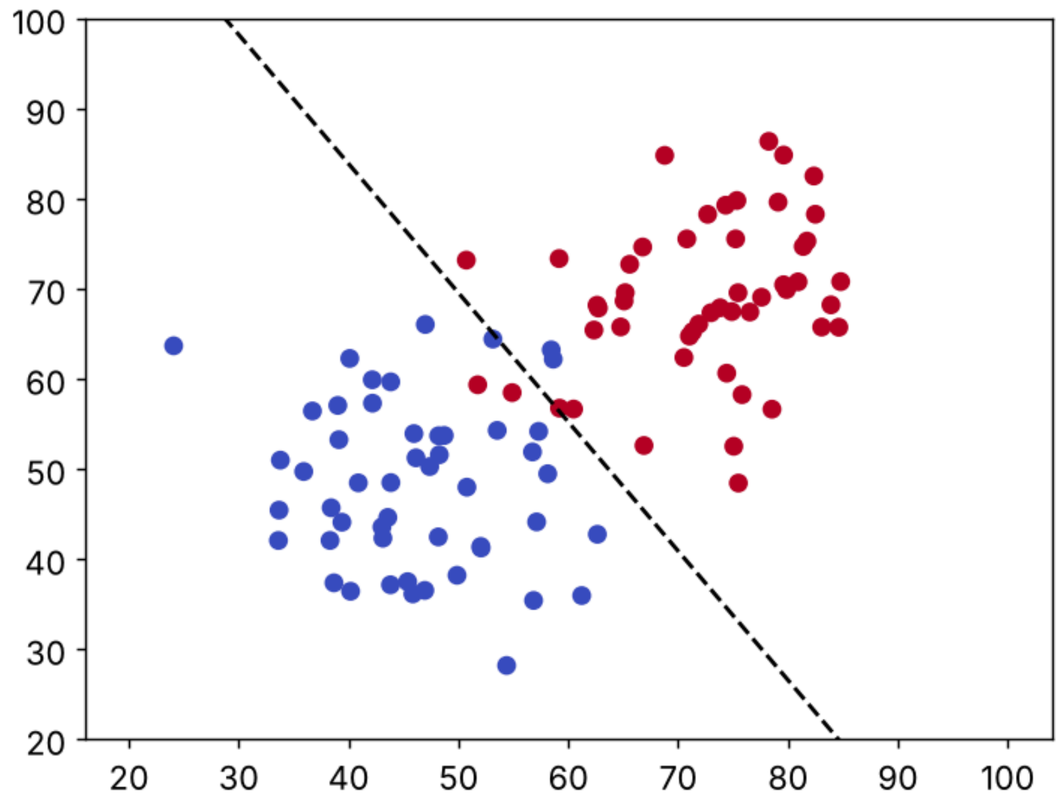
boundary: $w_1x_1 + w_2x_2 + b = 0$

- Fit the model, read w and b
- Solve the boundary equation for x_2
- Red = admitted, blue = not admitted

ml_demo.py

```python
# X: [exam 1, exam 2]   y: 1 = admitted
clf = LogisticRegression().fit(X, y)
w, b = clf.coef_[0], clf.intercept_[0]
x1 = np.linspace(20, 100, 50)
x2 = -(w[0] * x1 + b) / w[1]   # w.x + b = 0
plt.scatter(*X.T, c=y, cmap="coolwarm")
plt.plot(x1, x2, "k--")
plt.ylim(20, 100); plt.show()
```

</> OUTPUT



Log-Loss & Learning

Why Not Use MSE?

Choosing the right cost for probabilities

MSE + sigmoid = bumpy

Squared error through a sigmoid gives a non-convex cost with flat regions, so gradient descent can get stuck.

Weak penalties

With MSE, a confident wrong answer (p = 0.01 for a true 1) costs at most 1.

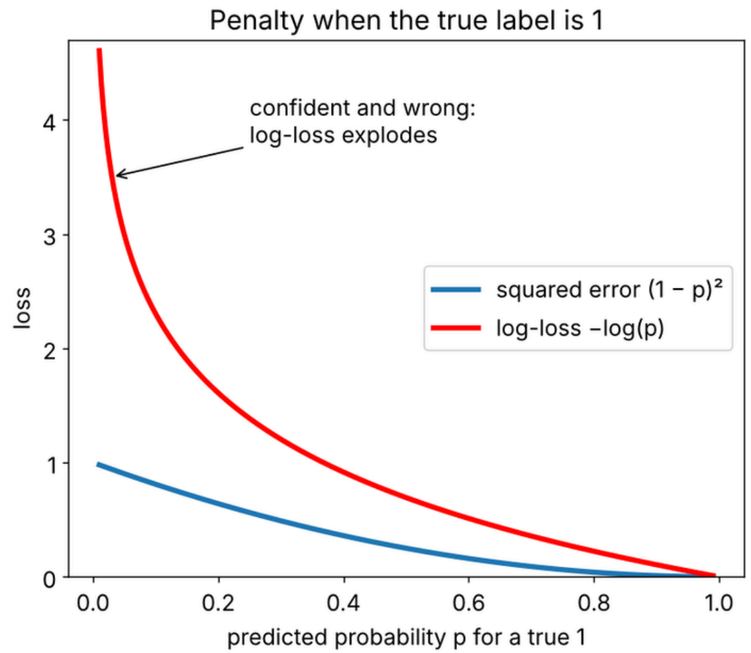
Log-loss instead

Log-loss punishes confident mistakes very hard and is convex for logistic regression: one bowl, one minimum.

Probability meaning

Minimizing log-loss is the same as finding the weights that make the observed labels most likely (maximum likelihood).

VISUAL



For a true label of 1, log-loss grows without limit as p → 0; squared error never goes above 1.

Log-Loss (Binary Cross-Entropy)

The cost function of logistic regression

Formula

$J = -(1/n) \Sigma [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$

When y = 1

Only -log(p) remains: the loss is small when p is close to 1.

When y = 0

Only -log(1 - p) remains: the loss is small when p is close to 0.

Perfect score

A perfect model has log-loss 0. Always predicting 0.5 gives log(2) ≈ 0.693.

FORMULA

```
loss = -[ y·log(p) + (1 - y)·log(1 - p) ]
```

Symbol	Meaning
y	true label, 0 or 1
p	predicted probability of class 1
log	natural logarithm

Example: y = 1, p = 0.9 → 0.105 (good) · y = 1, p = 0.1 → 2.303 (bad)

Log-Loss in Code

By hand and with scikit-learn

`sklearn.metrics.log_loss(y_true, p)`

- Compute the loss for each sample
- Confident mistakes dominate the total
- Same number as scikit-learn's log_loss

ml_demo.py

```python
from sklearn.metrics import log_loss
y = np.array([1, 1, 0, 0])
p = np.array([0.9, 0.6, 0.2, 0.7])     # predicted P(y = 1)
per_point = -(y * np.log(p) + (1 - y) * np.log(1 - p))
print("loss per point: ", per_point.round(3))
print("mean (by hand): ", per_point.mean().round(4))
print("sklearn log_loss:", round(log_loss(y, p), 4))
```

</> OUTPUT

```
loss per point:  [0.105 0.511 0.223 1.204]
mean (by hand):  0.5108
sklearn log_loss: 0.5108
```

Training with Gradient Descent

The same algorithm as linear regression

No closed formula

Unlike linear regression, there is no normal equation for logistic regression; we minimize log-loss iteratively.

The gradient

$\partial J/\partial w = (1/n)\ \Sigma\ (p_i - y_i) \cdot x_i$ and $\partial J/\partial b = (1/n)\ \Sigma\ (p_i - y_i)$.

Looks familiar

Same form as linear regression: prediction minus truth, times the input. Only $p = \sigma(w \cdot x + b)$ is new.

Update rule

$w \leftarrow w - \eta \cdot \partial J/\partial w$, $b \leftarrow b - \eta \cdot \partial J/\partial b$, repeated until the loss stops falling.

FORMULA

$$\partial J/\partial w = (1/n)\ \Sigma\ (p_i - y_i)\ x_i$$

Symbol	Meaning
$p_i - y_i$	error for sample i
η	learning rate
n	number of samples

Example: $p = 0.8$, $y = 0$, $x = 3$ → this sample pushes the gradient by $(0.8 - 0) \cdot 3 = 2.4$

Logistic Regression from Scratch

Pass or fail from hours studied

```
p = sigmoid(w*x + b); w -= lr * mean((p - y) * x)
```

- 50 students failed, 50 passed
- The loss falls as w and b improve
- Boundary = hours where p = 0.5

ml_demo.py

```python
rng = np.random.default_rng(0)
X = np.r_[rng.normal(2, 1, 50), rng.normal(5, 1, 50)]
y = np.r_[np.zeros(50), np.ones(50)]        # 1 = pass
w, b, lr = 0.0, 0.0, 0.1
for epoch in range(3001):
    p = 1 / (1 + np.exp(-(w * X + b)))
    w -= lr * np.mean((p - y) * X)
    b -= lr * np.mean(p - y)
    if epoch % 1500 == 0:
        L = -np.mean(y*np.log(p) + (1-y)*np.log(1-p))
        print(f"{epoch:4}: w={w:.2f} b={b:.2f} loss={L:.4f}")
print("p = 0.5 at", round(-b / w, 2), "hours")
```

</> OUTPUT

```
   0: w=0.07 b=0.00 loss=0.6931
1500: w=1.84 b=-6.35 loss=0.1865
3000: w=2.33 b=-8.16 loss=0.1622
p = 0.5 at 3.5 hours
```

The Learned Sigmoid

Probability of passing for every number of hours

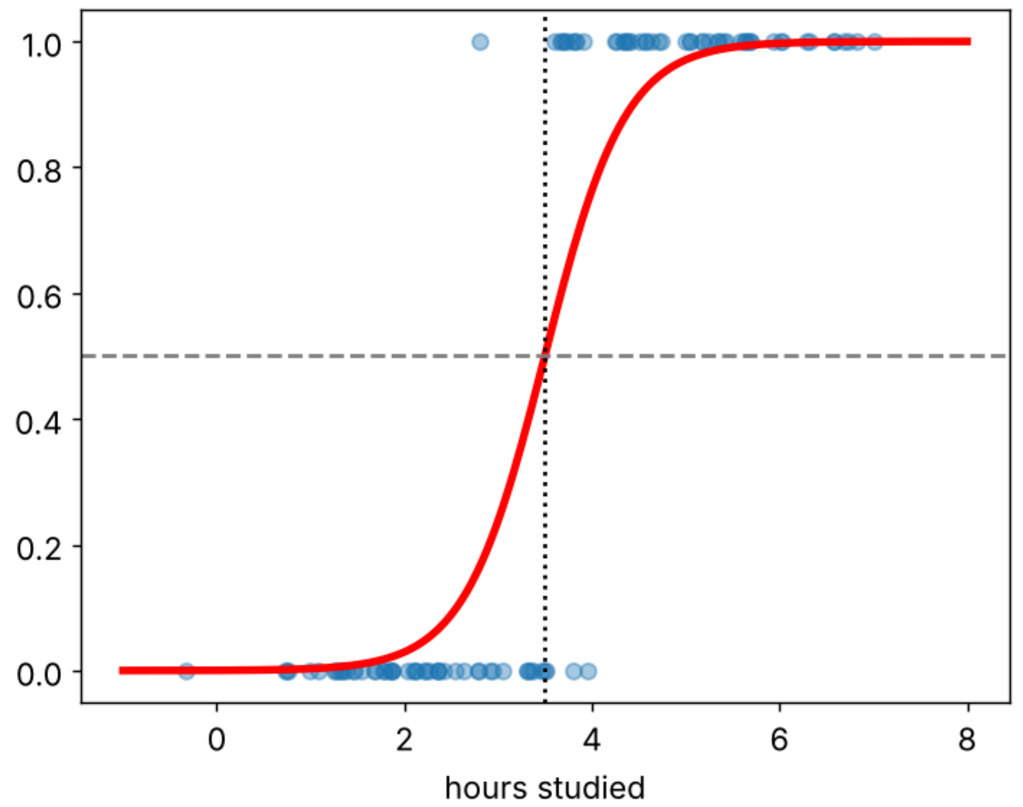
$$p(\text{hours}) = \sigma(w \cdot \text{hours} + b)$$

- Dots: students (0 = fail, 1 = pass)
- Red curve: predicted probability
- Dotted line: the 0.5 decision point

ml_demo.py

```python
# w, b learned on the previous slide
xs = np.linspace(-1, 8, 200)
p = 1 / (1 + np.exp(-(w * xs + b)))
plt.scatter(X, y, alpha=0.4)
plt.plot(xs, p, c="red", lw=3)
plt.axhline(0.5, ls="--", c="grey")
plt.axvline(-b / w, ls=":", c="k")
plt.xlabel("hours studied"); plt.show()
```

</> OUTPUT



Many Classes

Multiclass Classification

From two classes to many

One-vs-Rest (OvR)

Train one binary model per class ("this class or not"). Predict the class whose model gives the highest probability.

Softmax (multinomial)

One model gives a score to every class and softmax turns all scores into probabilities that sum to 1.

Cross-entropy loss

The multiclass log-loss: -log of the probability given to the true class.

In scikit-learn

LogisticRegression uses the softmax (multinomial) approach automatically when there are more than two classes.

One-vs-Rest vs. softmax

	One-vs-Rest	Softmax
Models trained	k binary	1 joint
Probabilities sum to 1	not always	always
Typical use	simple, any binary model	default for logistic

k = number of classes

The Softmax Function

Turning scores into class probabilities

Formula

$p_k = e^{z_k} / \Sigma_j e^{z_j}$: exponentiate every score, then divide by the total.

Properties

All probabilities are positive and add up to exactly 1; the largest score gets the largest probability.

Binary case

With two classes softmax gives the same result as the sigmoid.

Cross-entropy

Loss = -log(p of the true class). A confident correct answer costs almost 0.

FORMULA

$$p_k = e^{z_k} / \Sigma_j e^{z_j}$$

Symbol	Meaning
z_k	score of class k = $w_k \cdot x + b_k$
p_k	probability of class k
Σ_j	sum over all classes

Example: $z = [2, 1, 0.1] \rightarrow p \approx [0.66, 0.24, 0.10]$

Softmax in Code

Three classes, three probabilities

```
np.exp(z) / np.exp(z).sum()
```

- Subtract the max score for numerical stability
- argmax picks the predicted class
- Cross-entropy uses the true class's probability

ml_demo.py

```python
def softmax(z):
    e = np.exp(z - z.max())          # stable version
    return e / e.sum()

z = np.array([2.0, 1.0, 0.1])        # scores for 3 classes
p = softmax(z)
print("probabilities:", p.round(3), "  sum =", p.sum())
print("predicted class:", p.argmax())
ce = lambda k: round(-np.log(p[k]), 4)   # cross-entropy
print("loss if the true class is 0:", ce(0))
print("loss if the true class is 2:", ce(2))
```

</> OUTPUT

```
probabilities: [0.659 0.242 0.099]   sum = 1.0
predicted class: 0
loss if the true class is 0: 0.417
loss if the true class is 2: 2.317
```

Multiclass on the Iris Dataset

Three species, one logistic regression model

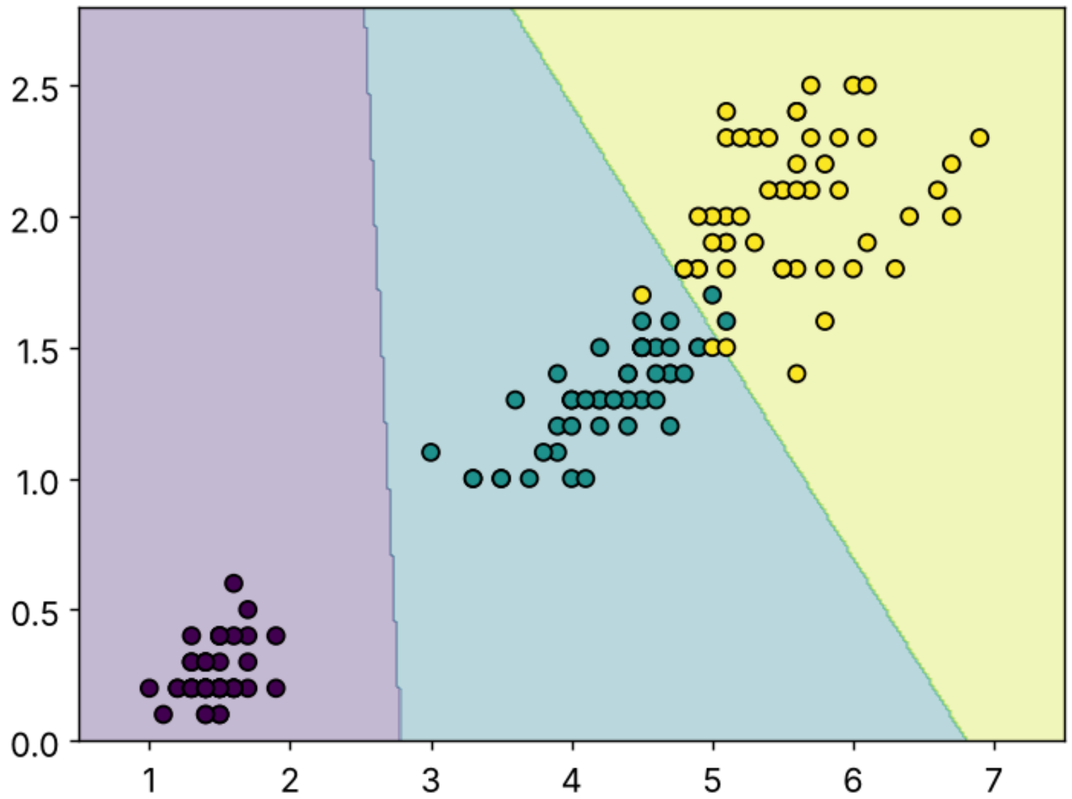
`LogisticRegression() with 3 classes`

- Two petal features, three species
- Each color region = one predicted class
- Boundaries are straight lines

ml_demo.py

```python
X, y = load_iris(return_X_y=True)
X = X[:, 2:]        # petal length, width
clf = LogisticRegression().fit(X, y)
a, b = np.meshgrid(np.linspace(.5, 7.5, 300),
                   np.linspace(0, 2.8, 300))
Z = clf.predict(np.c_[a.ravel(), b.ravel()])
Z = Z.reshape(a.shape)
plt.contourf(a, b, Z, alpha=0.3)
plt.scatter(*X.T, c=y, ec="k"); plt.show()
```

</> OUTPUT



Scoring the Results

The Confusion Matrix

Counting every kind of right and wrong answer

True positive (TP)
The patient is sick and the model says sick.

False positive (FP)
Healthy, but the model says sick: a false alarm (type I error).

False negative (FN)
Sick, but the model says healthy: a missed case (type II error).

True negative (TN)
Healthy and the model says healthy.

Rows = actual, columns = predicted

	Predicted 1	Predicted 0
Actual 1	TP	FN
Actual 0	FP	TN

Which error is worse depends on the problem: missing cancer (FN) is far worse than a false alarm (FP)

Classification Metrics

Why accuracy alone is not enough

Metric	Formula	Question it answers
Accuracy	(TP + TN) / all	How often is the model right overall?
Precision	TP / (TP + FP)	When it says positive, how often is it right?
Recall	TP / (TP + FN)	Of all real positives, how many did it find?
F1 score	2·P·R / (P + R)	One number balancing precision and recall
Log-loss	cross-entropy	How good are the probabilities themselves?

Imbalanced data? Look at precision, recall and F1, not just accuracy

Logistic Regression on Real Data

Breast cancer dataset from scikit-learn

`make_pipeline(StandardScaler(), LogisticRegression())`

- 569 tumors, 30 features each
- Scale features, then fit logistic regression
- predict_proba: [P(malignant), P(benign)]

ml_demo.py

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0)
model = make_pipeline(StandardScaler(), LogisticRegression())
model.fit(X_tr, y_tr)
print("test accuracy:", round(model.score(X_te, y_te), 3))
print(model.predict_proba(X_te[:3]).round(3))
```

</> OUTPUT

```
test accuracy: 0.982
[[0.989 0.011]
 [1.    0.   ]
 [0.998 0.002]]
```

Confusion Matrix & Classification Report

Precision, recall and F1 for every class

`classification_report(y_true, y_pred)`

- Confusion matrix rows: actual malignant, benign
- The report shows precision, recall and F1 per class
- support = number of test samples in that class

ml_demo.py

```python
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
pred = model.predict(X_te)
print(confusion_matrix(y_te, pred))
print(classification_report(y_te, pred,
      target_names=["malignant", "benign"]))
```

</> OUTPUT

```
[[40  2]
 [ 0 72]]
              precision    recall  f1-score   support

   malignant       1.00      0.95      0.98        42
      benign       0.97      1.00      0.99        72

    accuracy                           0.98       114
   macro avg       0.99      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

Choosing the Threshold

Trading precision against recall

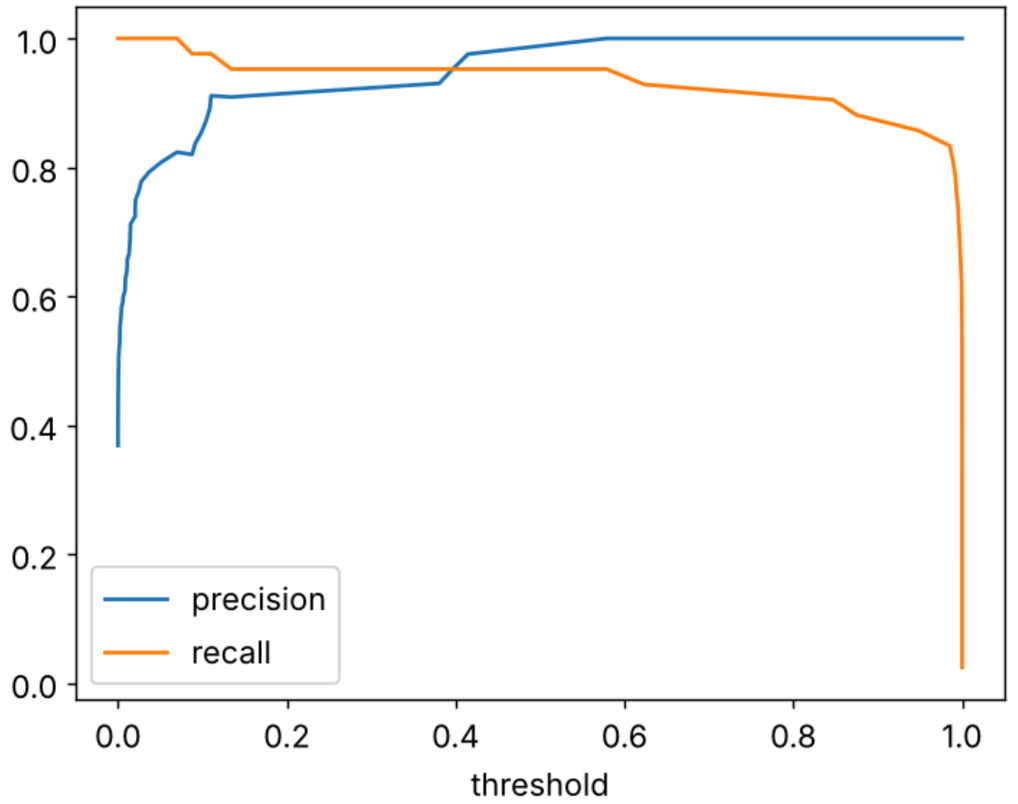
`predict 1 if p ≥ threshold (default 0.5)`

- Lower threshold: more recall
- Higher threshold: more precision
- Positive class here = malignant

ml_demo.py

```python
from sklearn import metrics
p_mal = model.predict_proba(X_te)[:, 0]
y_mal = (y_te == 0)
P, R, T = metrics.precision_recall_curve(
    y_mal, p_mal)
plt.plot(T, P[:-1], label="precision")
plt.plot(T, R[:-1], label="recall")
plt.xlabel("threshold"); plt.legend()
plt.show()
```

</> OUTPUT



ROC Curve & AUC

Performance across every threshold at once

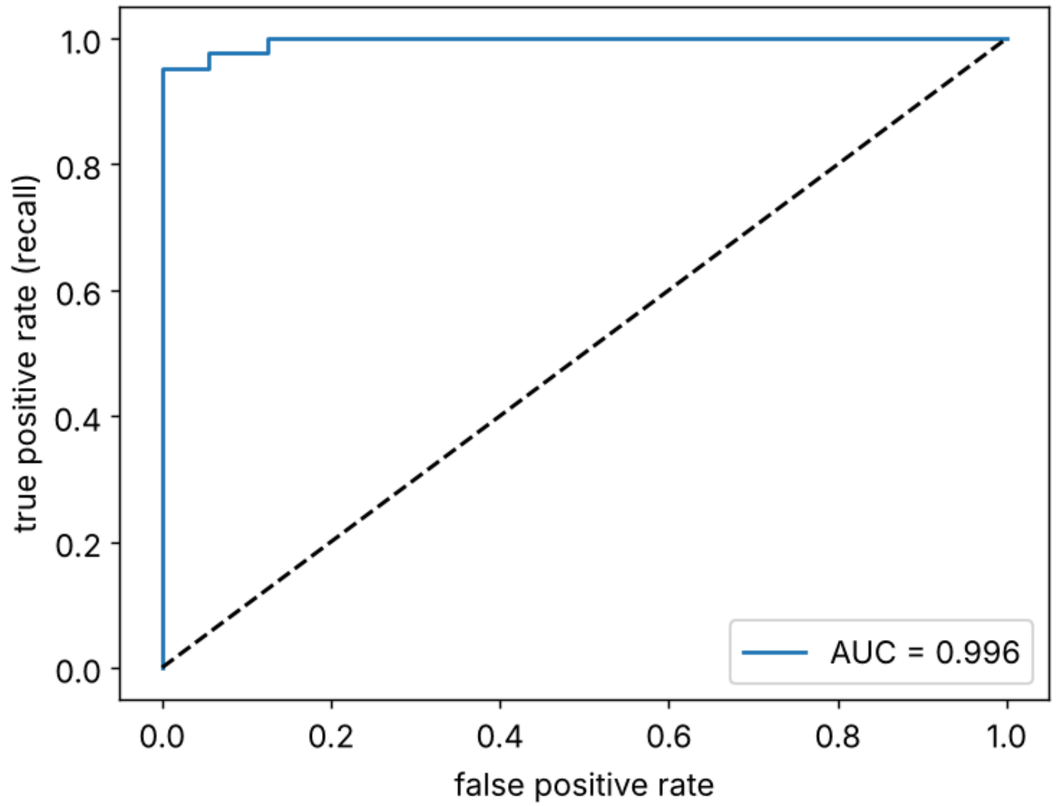
`roc_auc_score(y, p)` → 1.0 perfect, 0.5 random

- Each point = one threshold
- Closer to the top-left corner = better
- AUC = area under the curve

ml_demo.py

```python
fpr, tpr, _ = metrics.roc_curve(y_mal, p_mal)
auc = metrics.roc_auc_score(y_mal, p_mal)
plt.plot(fpr, tpr, label=f"AUC = {auc:.3f}")
plt.plot([0, 1], [0, 1], "k--")  # random
plt.xlabel("false positive rate")
plt.ylabel("true positive rate (recall)")
plt.legend(); plt.show()
```

</> OUTPUT



Regularization in Logistic Regression

Controlling overfitting with C

Built in

scikit-learn's LogisticRegression uses an L2 (Ridge) penalty by default.

C = 1 / α

C is the inverse of the strength: small C = strong penalty and simpler model, large C = weak penalty.

L1 option

penalty="l1" (with solver="liblinear" or "saga") sets some weights to 0, like Lasso.

Choose by validation

Try C = 0.01, 0.1, 1, 10, 100 and keep the best cross-validation score.

FORMULA

$$J = \text{log-loss} + (1 / C) \cdot \text{penalty}(w)$$

Symbol	Meaning
C	inverse regularization strength (default 1.0)
penalty	Σw^2 (L2) or $\Sigma \lvert w \rvert$ (L1)

Example: LogisticRegression(C=0.1) → smaller weights, smoother boundary

Linear vs. Logistic Regression

Same family, different jobs

	Linear regression	Logistic regression
Task	Regression: predict a number	Classification: predict a class
Output	Any real number	Probability between 0 and 1
Model	$\hat{y} = w \cdot x + b$	$p = \sigma(w \cdot x + b)$
Cost	MSE	Log-loss (cross-entropy)
Training	Normal equation or gradient descent	Gradient descent (iterative)
Metrics	MAE, RMSE, R^2	Accuracy, precision, recall, F1, AUC

Both are linear models: the difference is the output function and the loss

Thank You

Presented by TriNode Team

Best Wishes